

PRACTICA 5A

Víctor Rubio Rojo

Ismael Gonzalez Sañudo

1.

Para las pruebas unitarias utilizamos las técnicas de partición equivalente (dividiendo los posibles valores de entrada de los métodos en grupos válido y no válido) y AVL (Análisis de Valores Límite, mirando el comportamiento de los métodos con valores situados en los bordes de las particiones):

Para el método `precio()` de la clase `Seguro`:

Fecha de inicio:

No válido (futuro): Mañana (debería dar 0).

Válido < 1 año (20% descuento): Hoy (límite inferior), Hace 6 meses, Hace 1 año menos 1 día (límite superior).

Válido >= 1 año (sin descuento): Hace exactamente 1 año (límite inferior), Hace más de 1 año.

Potencia:

< 90 (sin multiplicador): 89 (límite superior).

[90, 110] (x1.05): 90 (límite inferior), 100 (nominal), 110 (límite superior).

> 110 (x1.2): 111 (límite inferior).

Cobertura: `TODO_RIESGO` (1000), `TERCEROS_LUNAS` (600), `TERCEROS` (400).

Para el método `totalSeguros()` de la clase `Cliente`:

Lista de seguros: Vacía (0), Con 1 seguro, Con varios seguros (>1).

Minusvalía: `True` (-25%), `False` (sin descuento).

Una vez diseñados y ejecutados los casos de prueba de caja negra, procedemos a medir la calidad de nuestras pruebas aplicando técnicas de caja blanca.

Para ello, utilizamos la herramienta **Eclemma** integrada en Eclipse:

- **Cobertura de Sentencias:** Asegurar que cada línea de código de los métodos bajo prueba se ejecuta al menos una vez.
- **Cobertura de Decisión/Condición:** Asegurar que cada condición lógica ha sido evaluada tanto a verdadero como a falso.

Comprobamos que los métodos con lógica de negocio (`precio()` en `Seguro` y `totalSeguros()` en `Cliente`) aparecen casi completamente en verde, a excepción de dos coberturas parciales: un `if`

y un switch, este último debido a un caso “default” del propio compilador (los 3 casos reales sí se comprueban). En el caso del if, se debe a que la segunda condición del ‘or’ no es alcanzable al estar implícita en la primera, por lo que la segunda sobra.

Arreglando este problema, conseguimos una cobertura del 90%, debido a que se han excluido del proceso de prueba los getters y setters, además del problema del caso default del switch.

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
> SegurosCommon	90,0 %	449	50	499

2.

Se ha realizado el diseño y ejecución de pruebas de integración.

Caja Negra: Se aplicó la técnica de Partición Equivalente basándonos en la base de datos H2 inicial, diseñando tres casos clave:

- Caso Válido 1 (Con seguros): DNI 11111111A. Verifica que se muestra el nombre Juan y se cargan sus 3 seguros.
- Caso Válido 2 (Sin seguros - Límite): DNI 33333333A. Verifica que se muestra el nombre Luis y la lista de seguros queda vacía (0 elementos).
- Caso No Válido (No existe): DNI 12345678Z. Verifica el manejo de errores, esperando el texto "Error en BBDD" y la lista vacía.

Para la depuración de esto, se ha visto que en VistaAgente.java el botón “Buscar” capturaba el nombre en vez del DNI. Se establecieron en el POM de SegurosGUI las dependencias de H2, Negocio y DAO, y se estableció correctamente ClientesDAO y SegurosDAO en el @BeforeEach del test para establecer la conexión real a la base de datos.

Caja Blanca:

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
> SegurosDAOH2	37,7 %	260	429	689
> SegurosCommon	35,5 %	177	322	499
> SegurosBusiness	8,1 %	14	159	173
> SegurosGUI	97,2 %	485	14	499

Usando EclEmma vemos un 97,2% de cobertura. Con el 100% en el flujo principal (la cobertura de Decisión/Condición) y ese 2,8% restante es debido a que no se alcanza nunca el catch (DataAccessException), lo cual es obvio ya que no debemos forzar un fallo de conexión física.